

Documento editable - Método de Newton Raphson

Estudiante: Andrey Felipe Pinto Uribe

Código: 0192177

Archivo fuente: Metodo_Newton_Raphson_0192177_Andrey_Felipe_Pinto_Uribe.py

1. Objetivo

El programa implementa el método de Newton Raphson para aproximar una raíz a partir de un valor inicial y de la derivada de la función.

2. Funcionamiento general del programa

- Cada fila muestra la nueva aproximación x , el valor $f(x)$, la derivada $f'(x)$ y el error.
- La fórmula usada es $x_{\text{nuevo}} = x - f(x) / f'(x)$.
- El método se detiene cuando el error o el valor absoluto de $f(x)$ cumple la tolerancia.
- Si la derivada fuera cero, el programa detendría el proceso para evitar una división por cero.

3. Pruebas o validaciones realizadas

- Se usa $f(x) = x^3 - x - 2$ y su derivada $f'(x) = 3x^2 - 1$.
- Se inicia desde $x_0 = 1.5$.
- Se valida la raíz aproximada, el número de iteraciones y $f(\text{raiz})$.

4. Evidencia del código fuente

A continuación se incluye el código fuente del archivo entregado. Este contenido es editable en Word.

```
1  """
2  Método de Newton Raphson
3  Estudiante: Andrey Felipe Pinto Uribe
4  Código: 0192177
5
6  Archivo preparado para entrega en Moodle. Contiene la implementación,
7  un ejemplo de ejecución y validaciones básicas de entrada.
8  """
9
10 from dataclasses import dataclass
11 from typing import Callable, List
12
13
```

```

14 @dataclass
15 class IteracionNewton:
16     numero: int
17     x: float
18     fx: float
19     dfx: float
20     error: float
21
22
23 @dataclass
24 class ResultadoNewton:
25     raiz: float
26     iteraciones: int
27     convergio: bool
28     historial: List[IteracionNewton]
29
30
31 def newton_raphson(
32     funcion: Callable[[float], float],
33     derivada: Callable[[float], float],
34     x0: float,
35     tolerancia: float = 1e-6,
36     max_iter: int = 100,
37 ) -> ResultadoNewton:
38     if tolerancia <= 0:
39         raise ValueError("La tolerancia debe ser positiva.")
40     if max_iter <= 0:
41         raise ValueError("El número máximo de iteraciones debe ser positivo.")
42
43     historial: List[IteracionNewton] = []
44     x = x0
45
46     for numero in range(1, max_iter + 1):
47         fx = funcion(x)
48         dfx = derivada(x)
49         if dfx == 0:

```

```

50         raise ZeroDivisionError("La derivada es cero; el metodo no puede continuar.")
51
52     x_nuevo = x - fx / dfx
53     error = abs(x_nuevo - x)
54
55     historial.append(
56         IteracionNewton(
57             numero=numero,
58             x=x_nuevo,
59             fx=funcion(x_nuevo),
60             dfx=derivada(x_nuevo),
61             error=error,
62         )
63     )
64
65     if error <= tolerancia or abs(funcion(x_nuevo)) <= tolerancia:
66         return ResultadoNewton(
67             raiz=x_nuevo,
68             iteraciones=numero,
69             convergio=True,
70             historial=historial,
71         )
72
73     x = x_nuevo
74
75     return ResultadoNewton(
76         raiz=historial[-1].x,
77         iteraciones=max_iter,
78         convergio=False,
79         historial=historial,
80     )
81
82
83 def imprimir_tabla(historial: List[IteracionNewton]) -> None:
84     encabezado = f"{'i':>3} {'x':>12} {'f(x)':>14} {'df(x)':>14} {'error':>12}"
85     print(encabezado)

```

```

86     print("-" * len(encabezado))
87     for paso in historial:
88         print(
89             f"{paso.numero:>3} "
90             f"{paso.x:>12.6f} "
91             f"{paso.fx:>14.6e} "
92             f"{paso.dfx:>14.6e} "
93             f"{paso.error:>12.6e}"
94         )
95
96
97 def funcion_ejemplo(x: float) -> float:
98     return x**3 - x - 2
99
100
101 def derivada_ejemplo(x: float) -> float:
102     return 3 * x**2 - 1
103
104
105 def main() -> None:
106     resultado = newton_raphson(funcion_ejemplo, derivada_ejemplo, 1.5)
107     print("Método de Newton-Raphson")
108     imprimir_tabla(resultado.historial)
109     print(f"\nRaiz aproximada: {resultado.raiz:.10f}")
110     print(f"Iteraciones: {resultado.iteraciones}")
111     print(f"Verificacion f(raiz): {funcion_ejemplo(resultado.raiz):.10e}")
112
113
114 if __name__ == "__main__":
115     main()

```

5. Evidencia de ejecución

La siguiente salida fue obtenida ejecutando el archivo Python de esta carpeta.

python Metodo_Newton_Raphson_0192177_Andrey_Felipe_Pinto_Uribe.py

Método de Newton-Raphson

i	x	f(x)	df(x)	error
1	1.521739	2.136928e-03	5.947070e+00	2.173913e-02
2	1.521380	5.893874e-07	5.943790e+00	3.593245e-04

Raiz aproximada: 1.5213798060

Iteraciones: 2

Verificacion f(raiz): 5.8938742598e-07

```
Run Metodo_Newton_Raphson_0192177_Andrey_Felipe_Pinto... x
C:\Users\andre\AppData\Local\Programs\Python\Python314\python.exe C:\Users\andre\Desktop\destok\numerical-analysis\entregas_moodle\newton_raphson\Metodo_Newton_Raphson_0192177_Andrey_Felipe_Pinto_Urbe.py
Método de Newton-Raphson
i      x      f(x)      df(x)      error
-----
1      1.521739  2.136928e-03  5.947070e+00  2.173913e-02
2      1.521380  5.893874e-07  5.943790e+00  3.593245e-04

Raiz aproximada: 1.5213798060
Iteraciones: 2
Verificacion f(raiz): 5.8938742598e-07

Process finished with exit code 0
```

6. Explicación de la salida

La salida confirma que el programa se ejecuta sin errores. Los valores impresos corresponden al ejemplo de prueba definido en el método `main`.

- Cada fila muestra la nueva aproximación x, el valor f(x), la derivada f'(x) y el error.
- La fórmula usada es $x_{\text{nuevo}} = x - f(x) / f'(x)$.
- El método se detiene cuando el error o el valor absoluto de f(x) cumple la tolerancia.
- Si la derivada fuera cero, el programa detendría el proceso para evitar una división por cero.

7. Conclusión

El programa cumple el objetivo del método numérico, presenta una estructura clara, valida condiciones importantes de entrada y muestra resultados coherentes para el caso de prueba utilizado.